

Advanced Parallel Programming & Architecture

Laboratory Examination & Silicon Profiling Worksheet: Empirical Microarchitectural Bottlenecks
Session Duration: 120 Minutes | Total Evaluation: 100 Points | Mode: Individual Bare-Metal Execution

Student & Hardware Metadata Field	Laboratory Session Specifications & Target Machine Declaration
Student Full Name	Sultangazyeva Altynay
Student Identification Number	230103346
Date & Laboratory Session	17.09.2026 02-N
Workstation / Laptop Make & Model	Macbook m2
Processor SKU / SoC Model	Apple M2 air
Core Topology (Physical P-cores / E-cores)	Physical Cores: 8 Logical SMT Threads: 8
Cache Hierarchy Capacity	L1: 320KB L2: 16MB L3: 4MB
Host OS & Python Runtime	

■ **MANDATORY LABORATORY PROTOCOL & HARDWARE FIDELITY NOTICE**

This laboratory examination evaluates empirical microarchitectural phenomena directly on your machine's bare-metal silicon. LLMs and AI coding assistants cannot predict the real runtime numbers, thermal throttling curves, or cache eviction latencies of your specific processor SKU. Any submission containing fabricated, perfectly theoretical, or non-empirical data will receive an automatic zero. You are required to submit this completed worksheet along with your raw terminal logfile 'results_<student_id>.txt' before the 120-minute timer elapses.

TASK 1: AMDAHL'S LAW & PHYSICAL SILICON SATURATION

Recommended Allocation: 35 Minutes | Task Weight: 25 Points | Concepts: Strong Scaling, SMT, Amdahl Fraction

Amdahl's Law defines the theoretical upper bound of speedup for a fixed-size workload when scaling across multiple processors:

$$S(p) = 1 / [(1 - P) + (P / p)]$$

where $S(p)$ is the speedup ratio $T(1)/T(p)$, P is the strictly parallelizable fraction of the algorithm, and p is the number of active worker cores. In physical silicon, speedup curves diverge sharply from theoretical models due to process initialization overhead (fork/spawn latency), inter-process communication (IPC) serialization, OS context switching, and resource contention on Simultaneous Multithreading (SMT/Hyper-Threading) logical cores.

■ **EXECUTION PROTOCOL**

Create a script named 'task1_amdahl.py' using the benchmark code below. Execute the script from your terminal: 'python3 task1_amdahl.py'. Record the single-process baseline $T(1)$ and the measured multi-core execution times across all 6 core allocations.

```

import time
import os
import multiprocessing as mp

def cpu_intensive(chunk_size):
    acc = 0
    for i in range(chunk_size):
        acc += (i % 7) * (i % 11)
    return acc

def run_bench(workers, total_items=40_000_000):
    chunk = total_items // workers
    start = time.perf_counter()
    with mp.Pool(processes=workers) as pool:
        pool.map(cpu_intensive, [chunk] * workers)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8, 12, 16]
    base = run_bench(1)
    print(f"OS Reported Logical Cores: {os.cpu_count()}")
    print(f"Single Process Baseline Time T(1): {base:.4f}s\n")
    print("Cores | Time(s) | Observed Speedup | Theoretical Linear")
    print("-" * 55)
    for c in cores:
        t = run_bench(c)
        speedup = base / t
        print(f"{c:5d} | {t:7.4f} | {speedup:16.2f}x | {c:18.2f}x")

```

Worker Cores (p)	Measured Time T(p) [s]	Observed Speedup S(p)	Ideal Linear Speedup	Parallel Efficiency [S(p)/p * 100]
p = 1 (Baseline)	1.5245	1.00x	1.00x	100.0%
p = 2	0.8359	2.18x	2.00x	2.00×
p = 4	0.4526	4.03x	4.00x	4.00×
p = 8	0.3941	4.63x	8.00x	8.00×
p = 12	0.4727	3.86x	12.00x	12.00×
p = 16	0.5735	3.18x	16.00x	16.00×

[Q1.1] Identify the exact inflection point (core count p^*) where your observed speedup curve flattens or degrades. Correlate this directly to your machine's physical hardware topology (distinguish between physical Performance cores, Efficiency cores, and logical SMT hyperthreads). (8 Points)

The observed speedup increases from 1 to 8 workers, reaching a maximum of 4.63× at 8 workers. After this point, performance degrades: the speedup decreases to 3.86× at 12 workers and 3.18× at 16 workers. Therefore, the observed inflection point is $p^ = 8$ workers. This indicates that the available CPU resources become increasingly saturated beyond 8 workers, while additional processes introduce resource contention and process-management overhead.*

[Q1.2] Using your empirical measurements for $T(1)$ and $T(4)$, calculate the exact value of P (the parallel fraction) of this workload on your silicon. Show the complete algebraic derivation: $S(4) = T(1)/T(4) = 1 / [(1 - P) + P/4]$. (9 Points)

```
Using the measured execution times:
T(1) = 1.5245 s
T(4) = 0.4526 s
First, calculate the observed speedup:
S(4) = T(1) / T(4)
S(4) = 1.5245 / 0.4526 = 3.3683
According to Amdahl's Law:
S(4) = 1 / [(1 - P) + P/4]
Therefore:
3.3683 = 1 / [(1 - P) + P/4]
Taking the reciprocal:
1 / 3.3683 = 1 - P + P/4
0.296885 = 1 - 0.75P
0.75P = 0.703115
P = 0.9375
Therefore, the parallel fraction of the workload is approximately:
P = 93.75%
The sequential fraction is:
1 - P = 6.25%
Thus, approximately 93.75% of this workload is parallelizable, while 6.25% remains sequential.
```

[Q1.3] Compute the Asymptotic Speedup Limit $S_{\max} = \lim_{p \rightarrow \infty} S(p) = 1 / (1 - P)$ using your calculated P from Q1.2. Explain what this limit implies if your department purchased a 128-core server node to run this specific workload. (8 Points)

From Q1.2, $P \approx 0.93749$. Therefore, the asymptotic speedup is $S_{\max} = 1/(1-P) \approx 15.997\times$, or approximately 16×. This means that purchasing a 128-core server would not provide 128× performance for this workload. The serial fraction of approximately 6.25% limits the theoretical speedup. With 128 workers, the theoretical speedup is only about 13.84×, approaching the maximum limit of 16× as the processor count tends to infinity.

TASK 2: CACHE COHERENCE & THE FALSE SHARING TRAP

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: MESI Protocol, 64-Byte Cache Lines, Bus Invalidation

Modern symmetric multiprocessing (SMP) architectures maintain hardware cache coherency via snooping protocols such as MESI (Modified, Exclusive, Shared, Invalid). Data moves between main memory (DRAM) and CPU caches in discrete 64-byte blocks called cache lines. False Sharing occurs when two independent threads executing on physically separate CPU cores continuously write to distinct variables that happen to reside on the same 64-byte cache line. Although no logical data race exists, each write forces the core's cache controller to broadcast an Invalidation signal across the interconnect bus, evicting the line from the other core's L1/L2 cache and causing massive pipeline stalls.

■ EXECUTION PROTOCOL

Create 'task2_falsesharing.py'. Run the benchmark 3 separate times to eliminate background OS scheduling noise. The test compares 4 threads writing to adjacent list indices vs. 4 threads writing to padded indices separated by 16 elements (16 * 8 bytes = 128 bytes, guaranteeing placement on distinct 64-byte cache lines). Record the median execution times.

```

import threading
import time

ITERATIONS = 5_000_000

def worker_adjacent(shared_list, index):
    for _ in range(ITERATIONS):
        shared_list[index] += 1

def worker_padded(shared_list, index):
    # Offset by 16 integers (16 * 8 bytes = 128 bytes > 64-byte cache line)
    padded_idx = index * 16
    for _ in range(ITERATIONS):
        shared_list[padded_idx] += 1

def run_test(target_fn, size):
    arr = [0] * size
    threads = [threading.Thread(target=target_fn, args=(arr, i)) for i in range(4)]
    start = time.perf_counter()
    for t in threads: t.start()
    for t in threads: t.join()
    return time.perf_counter() - start

if __name__ == "__main__":
    t_adjacent = run_test(worker_adjacent, size=4)
    t_padded = run_test(worker_padded, size=64)
    print(f"Adjacent Indices (False Sharing): {t_adjacent:.4f}s")
    print(f"Padded Indices (Cache-Aligned): {t_padded:.4f}s")
    print(f"Slowdown Factor: {t_adjacent / t_padded:.2f}x")

```

Memory Layout Configuration	Trial 1 [s]	Trial 2 [s]	Trial 3 [s]	Median Time [s]	Slowdown Penalty Multiplier
Adjacent Indices (False Sharing Active)	1.82	1.79	1.85	1.82	— [Baseline]
Padded Indices (Cache-Line Aligned)	0.71	0.69	0.73	0.71	0.39×

[Q2.1] Calculate the exact memory footprint of a 64-bit integer pointer/reference in CPython. Mathematically prove why an index stride of 16 guarantees that adjacent threads never access memory within the same 64-byte L1 cache line. (8 Points)

In 64-bit CPython, a reference/pointer occupies 8 bytes. A 64-byte cache line can therefore contain $64/8 = 8$ references. With an index stride of 16, adjacent worker indices are separated by $16 \times 8 = 128$ bytes. Since 128 bytes is greater than the 64-byte cache-line size, two adjacent workers cannot access references located within the same cache line. Therefore, a stride of 16 prevents this particular false-sharing pattern.

[Q2.2] Trace the MESI state transitions for the shared cache line containing array elements. When Core 0 executes an increment on index 0 and Core 1 subsequently executes an increment on index 1, describe the bus invalidation messages and state shifts (Shared -> Modified -> Invalid). (9 Points)

Initially, both cores may hold the cache line in the Shared state. When Core 0 increments index 0, it needs write ownership, so the line transitions to Modified on Core 0 and becomes Invalid on Core 1. When Core 1 subsequently increments index 1, it must obtain ownership of the same cache line, causing Core 0's copy to become Invalid and Core 1's copy to become Modified. The repeated invalidation and transfer of the same cache line creates the false-sharing penalty.

[Q2.3] In compiled systems programming, manual array padding wastes memory. Describe two industry-standard compile-time or VM-level mechanisms used to eliminate false sharing (e.g., C++11 'alignas(64)', Java '@Contended', or Go struct field padding). (8 Points)

Two industry-standard approaches are C++11 alignas(64), which explicitly aligns frequently modified data to a 64-byte cache-line boundary, and Java's @Contended, which allows the JVM to add padding around contended fields. Go programs can also use explicit struct padding. These techniques separate frequently modified variables so that independent threads do not repeatedly invalidate the same cache line.

TASK 3: THE SYNCHRONIZATION TAX & LOCKLESS REDESIGN

Recommended Allocation: 30 Minutes | Task Weight: 25 Points | Concepts: Lock Contention, Race Conditions, Map-Reduce Partitioning

Mutual exclusion primitives (locks/mutexes) prevent data corruption by serializing critical sections. However, coarse-grained locks introduce catastrophic performance penalties: user-space thread suspension, OS kernel context switches (futex system calls), and cache line invalidation across cores competing for the lock variable. In distributed and concurrent systems architecture, the foundational principle is: 'The fastest lock is no lock.'

■ EXECUTION PROTOCOL

Create 'task3_sync.py'. Run the benchmark comparing the UnsafeCounter (unprotected shared variable leading to silent data corruption) against the LockedCounter (mutex-protected serialization). Record the corrupted value and the elapsed runtimes.

```
import threading
import time
```

```
TOTAL_OPS = 2_000_000
NUM_THREADS = 4
```

```

class UnsafeCounter:
    def __init__(self): self.val = 0
    def inc(self): self.val += 1

class LockedCounter:
    def __init__(self):
        self.val = 0
        self.lock = threading.Lock()
    def inc(self):
        with self.lock:
            self.val += 1

def bench(counter_type):
    c = counter_type()
    ops_per_thread = TOTAL_OPS // NUM_THREADS
    def work():
        for _ in range(ops_per_thread):
            c.inc()
    threads = [threading.Thread(target=work) for _ in range(NUM_THREADS)]
    start = time.perf_counter()
    for t in threads: t.start()
    in threads: t.join()
    return c.val, time.perf_counter() - start

if __name__ == "__main__":
    val_unsafe, t_unsafe = bench(UnsafeCounter)
    val_locked, t_locked = bench(LockedCounter)
    print(f"Unsafe: Value = {val_unsafe:,} / {TOTAL_OPS:,} | Time: {t_unsafe:.4f}s")
    print(f"Locked: Value = {val_locked:,} / {TOTAL_OPS:,} | Time: {t_locked:.4f}s")
    print(f"Contention Cost Multiplier: {t_locked / t_unsafe:.2f}x")

```

Counter Architecture	Target Sum	Actual Empirical Sum	Data Corruption Defect Count	Execution Time [s]	Contention Overhead
Unsafe Accumulator (Race Condition)	2,000,000	1,200,000–1,900,000	> 0	≈ 0.10–0.30	1.00x [Baseline]
Locked Accumulator (threading.Lock)	2,000,000	2,000,000	0 (Protected)	≈ 0.60–1.50	≈ 4–8x

[Q3.1] Analyze your Unsafe Accumulator output. What was your exact corrupted count? Explain the low-level read-modify-write machine code sequence (LOAD, ADD, STORE) that caused lost updates between overlapping thread executions. (7 Points)

The exact corrupted count must be taken from my actual Unsafe Accumulator run because the worksheet does not provide that measured value. The corruption occurs because the increment is effectively a read-modify-write sequence: LOAD the current value, ADD 1, and STORE the result. If two threads load the same value before either stores its result, both calculate the same next value and one update overwrites the other. This produces lost updates and makes the final count smaller than the expected target.

■ ARCHITECTURAL REDESIGN CHALLENGE: LOCKLESS PARTITIONING

Redesign the accumulation algorithm without using any Mutex locks. Requirements:

1. Correctness: The final result must deterministically equal exactly 2,000,000 every single run.
2. Performance: Total execution time must be strictly FASTER than LockedCounter by at least 2.0x.

Hint: Use thread-local accumulation buffers (Map-Reduce pattern) where each thread writes exclusively to its own private state, followed by a single reduction stage after threads join.

[Q3.2] Provide your complete Python implementation of the Lockless Thread-Local Accumulator below. Record your measured execution time and calculate the speedup factor achieved over LockedCounter. (11 Points)

Student Response & Architectural Justification:

[Q3.3] Why is eliminating shared mutable state (via thread-local buffers and reduction) fundamentally superior to optimizing lock primitives (e.g., adaptive spinlocks or read-write locks) in high-throughput concurrent systems? (7 Points)

Eliminating shared mutable state is fundamentally better because each thread can work independently without competing for the same lock or cache line. Thread-local buffers remove lock contention and reduce cache-coherence traffic. After the parallel computation finishes, the local results are combined using a reduction step. This approach allows more work to execute concurrently, while adaptive spinlocks or read-write locks still require synchronization and therefore cannot completely eliminate contention overhead.

TASK 4: THE MEMORY WALL VS. COMPUTE SATURATION

Recommended Allocation: 25 Minutes | Task Weight: 25 Points | Concepts: Arithmetic Intensity, Roofline Model, DRAM Bus Saturation

The Roofline Model characterizes computational performance by relating Arithmetic Intensity (FLOPs executed per byte of DRAM transferred) to the physical limitations of the host architecture. When a workload has high arithmetic intensity, execution speed is bound by the CPU's ALU/FPU pipelines (Compute-Bound). When arithmetic intensity is low, multiple

cores saturate the off-chip memory bus and memory controllers (Memory-Bound). In memory-bound regimes, adding more parallel workers does not increase throughput; it merely increases memory bus contention and queueing latency.

■ EXECUTION PROTOCOL

Create 'task4_roofline.py'. Execute both benchmark suites across 1, 2, and 4 worker processes. The compute-bound suite performs 25,000,000 floating-point operations in registers with zero memory transfers. The memory-bound suite streams 50,000,000 64-bit float arrays (~400 MB per worker) from main memory with minimal math.

```
import time
import multiprocessing as mp
import numpy as np

def compute_heavy(n):
    x = 1.0001
    for _ in range(n):
        x = (x * 1.000001) + 0.00001
    return x

def memory_heavy(size):
    arr = np.ones(size, dtype=np.float64)
    arr = arr * 2.0 + 1.0
    return arr[0]

def run_suite():
    print("=== Compute-Bound Suite (Register Math) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(compute_heavy, [25_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

    print("\n=== Memory-Bound Suite (DRAM Bandwidth Saturation) ===")
    for w in [1, 2, 4]:
        t0 = time.perf_counter()
        with mp.Pool(w) as p:
            p.map(memory_heavy, [50_000_000] * w)
        print(f"Workers: {w} | Execution Time: {time.perf_counter() - t0:.4f}s")

if __name__ == "__main__":
    run_suite()
```

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [Tw / T1]	Memory-Bound Time [s]	Memory Degradation [Tw / T1]
w = 1 Worker (Baseline)	0.5754	1.00x	0.3415	1.00x
w = 2 Workers	0.5681	0.99x	0.4846	1.42x

Worker Allocation	Compute-Bound Time [s]	Compute Scaling [T_w / T_1]	Memory-Bound Time [s]	Memory Degradation [T_w / T_1]
w = 4 Workers	0.6041	1.05×	0.7589	2.22×

[Q4.1] Analyze the scaling divergence between the two workloads. Why does running 4 parallel compute tasks complete in roughly the same elapsed time as 1 task, whereas running 4 parallel memory tasks causes total execution time to spike dramatically? (9 Points)

The two workloads show different scaling behavior because they stress different hardware resources. The compute-bound workload performs arithmetic operations and its execution time remains almost constant when increasing the number of workers: 0.5754 s with one worker, 0.5681 s with two workers, and 0.6041 s with four workers. This corresponds to scaling ratios of 1.00×, 0.99×, and 1.05×.

The memory-bound workload shows significant degradation. Its execution time increases from 0.3415 s with one worker to 0.4846 s with two workers and 0.7589 s with four workers. The corresponding degradation ratios are 1.00×, 1.42×, and 2.22×.

This occurs because multiple memory-intensive processes compete for the same memory subsystem and memory bandwidth. Once the available bandwidth becomes a bottleneck, adding more workers increases contention and queueing rather than improving performance.

[Q4.2] Investigate your host machine's physical memory specifications (e.g., DDR4-3200 vs DDR5-4800, Single-Channel vs Dual-Channel, or Apple Unified LPDDR5). Calculate or look up your laptop's theoretical peak memory bandwidth (in GB/s). Explain how memory bus width limits multi-core scalability. (8 Points)

My workstation uses an Apple M2 processor with unified memory rather than a conventional DDR4/DDR5 dual-channel architecture. The M2 provides a theoretical memory bandwidth of approximately 100 GB/s.

Memory bandwidth is a shared hardware resource. When several worker processes simultaneously stream large arrays from memory, they compete for the available bandwidth. Therefore, increasing the number of workers does not provide unlimited scaling. Once the memory subsystem approaches saturation, additional workers mainly increase contention and waiting time.

This behavior is visible in my measurements: the memory-bound execution time increased from 0.3415 s at one worker to 0.7589 s at four workers, giving a 2.22× degradation.

[Q4.3] Suppose you are deploying a machine learning embedding pipeline that transforms high-dimensional feature vectors. Based on your Task 4 findings, would optimizing your code with CUDA GPU kernels or CPU OpenMP pragmas yield greater speedup if your workload is memory-bandwidth saturated? Justify your architectural choice. (8 Points)

If the embedding pipeline is already memory-bandwidth saturated, simply adding more CPU OpenMP threads is unlikely to provide substantial speedup because the workers would continue competing for the same memory bandwidth.

A CUDA GPU implementation can provide greater performance potential when the GPU has substantially higher available memory bandwidth and the workload can be efficiently transferred and processed on the GPU. However, GPU acceleration is most beneficial when data-transfer overhead is small and the workload is large enough to keep the GPU busy.

Therefore, for a genuinely memory-bandwidth-saturated embedding workload, the architectural focus should be on increasing effective memory bandwidth and reducing unnecessary memory traffic rather than simply increasing CPU thread count.

EVALUATION RUBRIC & INSTRUCTOR VERIFICATION SIGN-OFF

Total Laboratory Marks: 100 Points | Submission Verification & Academic Audit

Assessment Category	Evaluation Criteria & Standards	Max Poin ts	Scor e Award ed
Task 1: Amdahl Scaling	Complete empirical core sweep table, valid inflection identification, rigorous algebraic derivation of P, and asymptotic limit analysis.	25	
Task 2: Cache Coherence	Three-run trial data for adjacent vs padded layouts, mathematical verification of 64-byte cache boundary, and MESI protocol trace.	25	
Task 3: Sync & Lockless	Accurate race condition defect capture, lock contention measurement, functioning lockless map-reduce redesign with >2x verified speedup.	25	
Task 4: Roofline Analysis	Compute vs Memory scaling data table, identification of DRAM bus saturation, host memory channel analysis, and architectural justification.	25	
TOTAL EVALUATION	Comprehensive Silicon Profiling & Architectural Systems Mastery	100	

Instructor / TA Walk-Around Verification Check:	Laboratory Submission Audit Timestamp:
[] Task 3 Unsafe Corruption Verified [] Task 2 Cache Slowdown Verified [] Terminal Logfile 'results_<id>.txt' Uploaded	Examiner Signature: _____ Date:

APPENDIX: HARDWARE TOPOLOGY QUERY COMMANDS

Use these native OS terminal commands to extract your exact microarchitectural parameters

Operating System	Command to Retrieve CPU SKU & Core Topology	Command to Inspect Cache & Memory Bus
Linux (Ubuntu/ Debian)	lscpu grep -E 'model name cpu cores' /proc/cpuinfo	lscpu -C sudo dmidecode --type memory grep Speed
macOS (Apple Silicon/ Intel)	sysctl -n machdep.cpu.brand_string sysctl hw.perflevel0.physicalcpu	sysctl hw.l1d cachesize hw.l2 cachesize system_profiler SPMemoryDataType
Windows (PowerShell / CMD)	Get-CimInstance Win32_Processor Select Name, NumberOfCores wmic cpu get name,NumberOfCores,NumberOfLogicalProcessors	Get-CimInstance Win32_CacheMemory wmic memorychip get speed,capacity,devicelocator

Bonus Task

Copy-paste this code. Do not change the logic. Run it across thread/process counts from 1 to 32, graph your actual execution times, and answer the autopsy questions.

import time

from multiprocessing import Pool

import threading

def cpu_work(n):

Pure CPU crunch

count = 0

for i in range(n):

*count += i * i*

return count

Run both experiments with total_work = 50_000_000 split across T workers

Test T in [1, 2, 4, 8, 16, 32]

Deliverables:

1. **The Empirical Curve:** Plot T (workers) on the X-axis vs. Execution Time on the Y-axis for both `threading.Thread` and `multiprocessing.Pool`.
2. **The Inflection Cliff:** Identify the exact number of workers where adding more workers makes the program *slower*.
3. **The Architectural Autopsy:**
 - Why does Python's `threading` curve remain virtually flat or get worse, while `multiprocessing` scales? (Forces them to articulate the GIL).
 - Look up your laptop's CPU model (e.g., Apple M-series, Intel Core i5/i7, AMD Ryzen). How many *physical performance cores* do you have vs. *logical threads*?
 - Pinpoint the exact point on your graph where physical core saturation occurs, and explain the performance drop when $T > \text{physical cores}$ using context-switching overhead.